

# **(Optional) Views and Controllers**

From HTML strings to Blade templates.

# Learning Goals

- Replace HTML strings with Blade views: `view()` , `{{ }}` , layouts, and partials.
- Define CRUD pages with `Route::resource` and named routes.
- Show validation errors and old input in a form, and redirect with a flash message.

So far, every answer was JSON. This optional lesson adds the **View** of MVC: pages that people read in a browser.

- In `6-2`, the controller built HTML as a **string**.
- Here, the controller only gets the data, and a **Blade template** builds the page.
- We keep the same `studentdb` database and the same `students` table.
- You can skip this lesson. The rest of the module builds APIs.

# Why Not HTML Strings?

```
public function index()  
{  
    $html = '<h1>All Students</h1>';  
    foreach (Student::all() as $student) {  
        $html .= '<p>' . e($student->name) . '</p>';  
    }  
    return $html;  
}
```

- Logic and HTML are mixed in one place.
- A designer cannot change the page without touching PHP.
- You must remember to escape every value yourself.

# The Same Page with a View

```
public function index()  
{  
    return view('students.index', ['students' => Student::all()]);  
}
```

```
@foreach ($students as $student)  
    <p>{{ $student->name }}</p>  
@endforeach
```

- `view('students.index', ...)` loads `resources/views/students/index.blade.php`. The dot means a folder.
- The second argument is the data the page can use.
- `{{ }}` escapes the value for you.

# Blade in One Table

| Blade                                                               | Meaning                                 |
|---------------------------------------------------------------------|-----------------------------------------|
| <code>{{ \$x }}</code>                                              | Print <code>\$x</code> , <b>escaped</b> |
| <code>@if ... @elseif ... @endif</code>                             | Conditions                              |
| <code>@foreach ... @endforeach</code>                               | Loop                                    |
| <code>@forelse ... @empty ...<br/>@endforelse</code>                | Loop, with a part for<br>an empty list  |
| <code>@extends</code> , <code>@section</code> , <code>@yield</code> | Page layouts                            |
| <code>@include('students._form')</code>                             | Reuse a piece of a<br>page              |

| Blade                                             | Meaning                                   |
|---------------------------------------------------|-------------------------------------------|
| <code>@csrf , @method( 'PUT' )</code>             | Form security and HTTP method             |
| <code>@error( 'name' ) ,<br/>old( 'name' )</code> | Validation messages and typed values      |
| <code>{{-- comment --}}</code>                    | A comment that is not sent to the browser |

- A Blade file is HTML with these extra tags. Laravel turns it into PHP.

# Escaping Protects Your Users

```
{{ $student->name }}      <!-- safe: a name like <script> is shown as text -->
{!! $student->name !!}     <!-- NOT escaped: runs the HTML or script -->
```

- Use `{{ }}` for everything that comes from users or the database.
- Use `{!! !!}` only for HTML that you wrote yourself.
- This is the same protection as `e()` in `6-2`, now automatic.



# Layouts: Write the Frame Once

resources/views/layouts/app.blade.php :

```
<html>
<head><title>@yield('title', 'Students')</title></head>
<body>
    @if (session('success'))
        <div class="alert alert-success">{{ session('success') }}</div>
    @endif
    @yield('content')
</body>
</html>
```

- The layout has the page frame: title, navigation, messages.
- `@yield('content')` is a hole that each page fills.

A page **extends** the layout and fills the holes:

```
@extends('layouts.app')  
  
@section('title', 'All Students')  
  
@section('content')  
    <h1>All Students</h1>  
    ...  
@endsection
```

- Change the layout once, and every page changes.
- The four pages in this lesson are short because the layout does the repeated work.

## Partials: Reuse a Form

The create page and the edit page show the same fields. They share one file:

```
{{-- create.blade.php --}}  
<form method="POST" action="{{ route('students.store') }}">  
    @csrf  
    @include('students._form', ['button' => 'Create'])  
</form>
```

- `@include` inserts `students/_form.blade.php` and passes `button` to it.
- The file name starts with `_` by convention: it is a piece, not a page.
- `edit.blade.php` uses the same partial with `['button' => 'Update']`.

# Resource Routes: Seven Routes in One Line

```
Route::resource('students', StudentController::class);
```

| Method | URL                      | Controller method |         |
|--------|--------------------------|-------------------|---------|
| GET    | /students                | index             | student |
| GET    | /students/create         | create            | student |
| POST   | /students                | store             | student |
| GET    | /students/{student}      | show              | student |
| GET    | /students/{student}/edit | edit              | student |
| PUT    | /students/{student}      | update            | student |
| DELETE | /students/{student}      | destroy           | student |

## Named Routes: No Hard-Coded URLs

```
<a href="{{ route('students.show', $student) }}">View</a>  
<a href="{{ route('students.edit', $student) }}">Edit</a>
```

- `route('students.show', $student)` builds `/students/3` from the name and the model.
- If you change a URL later, the links still work.
- The web routes ( `/students` ) and the API routes ( `/api/students` ) do not clash: the API routes have the `/api` prefix.

# Validation Errors in a Form

```
$request->validate([
    'name' => 'required|string|max:100',
    'age'   => 'required|integer|min:1|max:120',
    // ...
]);
```

If the data is wrong, Laravel **redirects back** and gives the page two things:

- `$errors` : the messages. `@error('age') ... {{ $message }}` `@enderror` shows one.
- Old input: `old('age')` returns what the user typed, so the form is not empty again.

```
<input type="number" name="age"
      value="{{ old('age', $student->age ?? '') }}"
      class="form-control @error('age') is-invalid @enderror">
@error('age')
    <div class="invalid-feedback">{{ $message }}</div>
@enderror
```

- `old('age', $default)` : the typed value, or the saved value when the form first opens.
- The API controller returns `422` JSON for the same mistake. The web controller shows the message on the page.



# Redirect and Flash Message

```
public function store(Request $request)
{
    $validated = $request->validate([ ... ]);
    Student::create($validated);

    return redirect()->route('students.index')
        ->with('success', 'Student created.');
```

- After a `POST`, **redirect** to a `GET` page. Then a browser reload does not send the form again.
- `->with('success', ...)` keeps the message in the session for **one** request.
- The layout shows it with `session('success')`.

# Two Things Every Form Needs

```
<form method="POST" action="{{ route('students.update', $student) }}">
    @csrf
    @method('PUT')
    ...
</form>
```

- `@csrf` adds a hidden **CSRF token**. A web `POST` without it gets **419 Page Expired**.
- HTML forms can only send `GET` and `POST`.  
`@method('PUT')` adds a hidden field, and Laravel treats the request as `PUT`. `@method('DELETE')` works the same way.
- API routes do not need either: they use tokens (Lesson 5) instead of sessions.

## Apply This to Your Project

- Put the page frame in one layout and fill it with `@yield` .
- Keep controllers short: get the data, then `return` `view(...)` .
- Use `{{ }}` for all user data. Use `@csrf` in every form.
- Use `Route::resource` and named routes for CRUD pages.
- Next: **6-4. (Optional) run1-6-3.sh** builds and runs this project.